RISC-V 5-Stage Pipelined Processor

Complete Study Guide

Comprehensive Technical Reference - All 5 Pipeline Stages

Fetch (IF) - Decode (ID) - Execute (EX) - Memory (MEM) - Writeback (WB)

Table of Contents

Section 1: Fetch Stage (IF) - Overview

The Fetch Cycle (IF) is the first stage of the RISC-V pipelined processor. Its primary job is to read the next instruction from memory and prepare it for decoding. The Fetch stage operates with four key sub-modules:

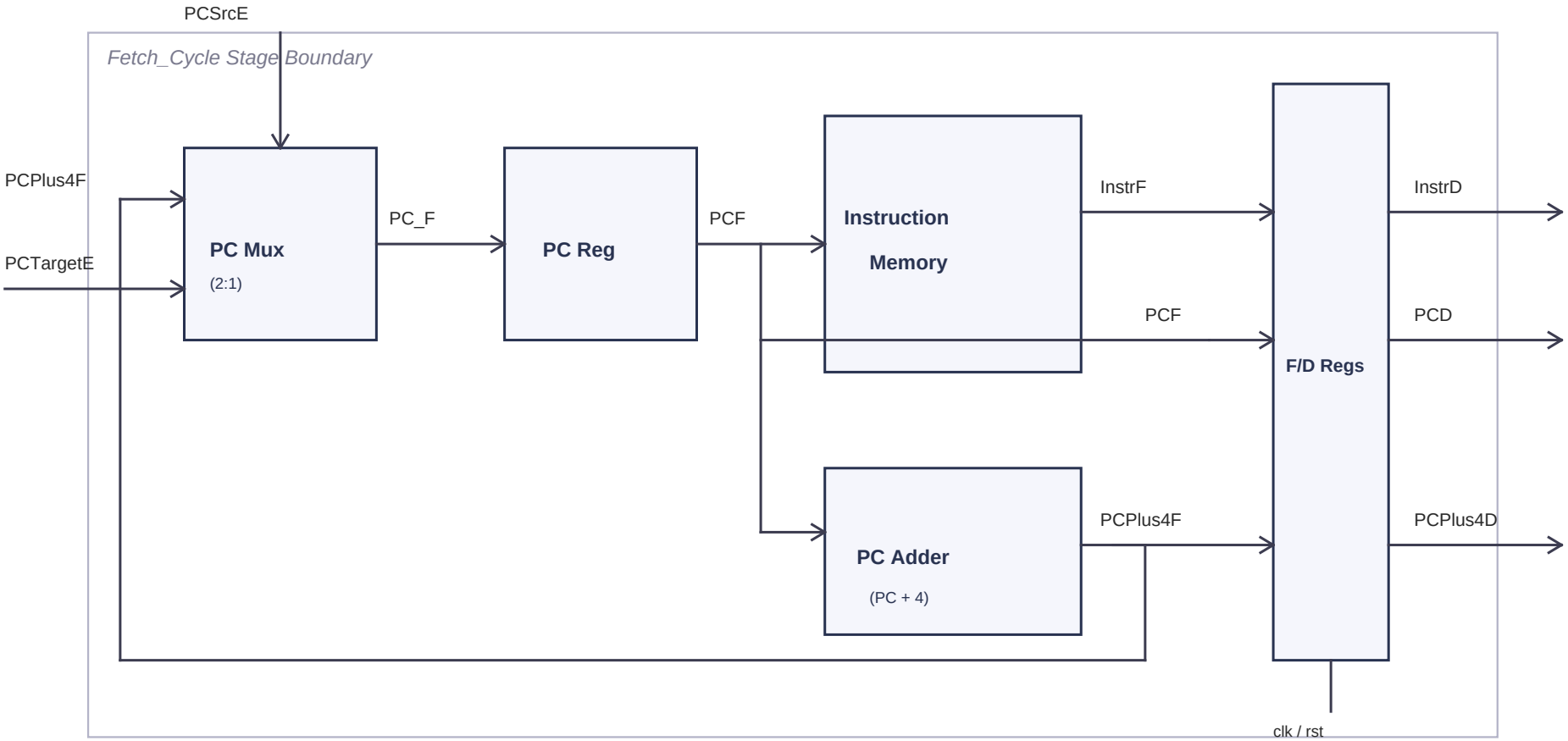
1. PC Mux (2:1 Multiplexer): Selects between the sequential address (PC+4) and a branch/jump target address (PCTargetE).
2. PC Module (Program Counter Register): A 32-bit synchronous register that stores the current instruction address.
3. Instruction Memory (IMEM): A read-only memory that outputs the 32-bit instruction word at the address given by PC.
4. PC Adder: A simple combinational adder that calculates PC + 4 for the next sequential instruction.

At each clock edge, the Fetch stage captures the current instruction (InstrF), the current PC (PCF), and PC+4 (PCPlus4F) into Fetch-to-Decode (F/D) pipeline registers, producing outputs InstrD, PCD, and PCPlus4D for the Decode stage.

Fetch Stage Interface Summary

Signal	Direction	Bit Width	Description
clk	Input	1	System clock
rst	Input	1	Active-low synchronous reset
PCSrcE	Input	1	PC mux select: 0=PC+4, 1=PCTargetE (from Execute stage)
PCTargetE [31:0]	Input	32	Branch/jump target address computed in Execute stage
InstrD [31:0]	Output	32	Fetched instruction registered into Decode stage
PCD [31:0]	Output	32	PC value registered into Decode stage
PCPlus4D [31:0]	Output	32	PC+4 value registered into Decode stage

Fetch Stage Block Diagram



Fetch Sub-Module Deep Dives

PC_Module: Program Counter Register

The PC_Module is a 32-bit synchronous register driven by posedge clk. On reset (rst == 0), it loads the value 0x00000000 (the first instruction address). Otherwise, it captures PC_Next (the output of the PC Mux) at every rising clock edge. This register provides the current instruction address (PCF) that feeds both the Instruction Memory address input and the PC Adder.

Key insight: The PC updates synchronously on the rising edge, meaning the new instruction address is stable before the next clock cycle begins. The reset is checked synchronously (inside the posedge clk block), so the reset takes effect on the next clock edge.

PC_Adder: Sequential Address Calculator

The PC_Adder is a simple combinational adder: c = a + b. In the Fetch stage, it is instantiated with a = PCF (current PC) and b = 32'h00000004 (the constant 4). Since RISC-V uses 32-bit (4-byte) instructions, incrementing by 4 advances to the next word-aligned instruction. The output PCPlus4F feeds back to the PC Mux (input a, selected when PCSrcE = 0) and also passes through the F/D pipeline registers as PCPlus4D.

Key insight: This adder has no clock or reset - it is purely combinational. The output is valid as soon as PCF changes.

Instruction_Memory: ROM with Hex File Loading

The Instruction Memory is a 1024-word x 32-bit array. It uses asynchronous read: RD = mem[A[31:2]]. The word-aligned addressing uses A[31:2] (discarding the 2 LSBs) because each 32-bit word occupies 4 byte addresses. For example, PC=0x00 reads mem[0], PC=0x04 reads mem[1], PC=0x08 reads mem[2].

On reset (rst == 0), the output is forced to 32'd0. At simulation startup, $readmemh("memfile.hex", mem) loads the program from a hexadecimal file. Each line in the hex file represents one 32-bit instruction word.

Key insight: The asynchronous read means the instruction is available in the same cycle that PC provides the address - no additional clock cycle is needed for the read.

2:1 Mux (PC Mux): Branch/Jump Selection

The Mux module implements c = (~s) ? a : b. In the Fetch stage:
- Input a = PCPlus4F (sequential next address)
- Input b = PCTargetE (branch/jump target from Execute stage)
- Select s = PCSrcE (from Execute stage: 1 if branch taken or jump)

When PCSrcE = 0 (no branch/jump), the mux selects PCPlus4F for normal sequential execution. When PCSrcE = 1 (branch taken or unconditional jump), the mux redirects to PCTargetE.

F/D Pipeline Registers: Fetch-to-Decode Boundary

The F/D pipeline registers capture three values at each posedge clk (or negedge rst for async reset):
- InstrD <= InstrF (the fetched instruction)
- PCD <= PCF (the PC of the fetched instruction)
- PCPlus4D <= PCPlus4F (PC+4 for potential use in link register or further pipeline)

On reset, all three are cleared to zero. These registers create the boundary between the Fetch and Decode stages, ensuring that each stage can operate on different instructions simultaneously.

Fetch Stage: Example Walkthrough

memfile.hex Contents:

```
@00000000
00500293   // addi x5, x0, 5
00300313   // addi x6, x0, 3
006283B3   // add  x7, x5, x6
00002403   // lw   x8, 0(x0)
00100493   // addi x9, x0, 1
00940533   // add  x10, x8, x9
```

Cycle-by-Cycle Fetch Trace (assuming PCSrcE = 0, no branches taken):

Cycle	PCF (Address)	mem Index [A[31:2]]	InstrF (Fetched)	Decoded Assembly	PCPlus4F
1	0x00000000	0	0x00500293	addi x5, x0, 5	0x00000004
2	0x00000004	1	0x00300313	addi x6, x0, 3	0x00000008
3	0x00000008	2	0x006283B3	add x7, x5, x6	0x0000000C
4	0x0000000C	3	0x00002403	lw x8, 0(x0)	0x00000010
5	0x00000010	4	0x00100493	addi x9, x0, 1	0x00000014
6	0x00000014	5	0x00940533	add x10, x8, x9	0x00000018

Walkthrough Explanation:

- Cycle 1: After reset, PC starts at 0x00. The Instruction Memory reads mem[0] = 0x00500293 (addi x5, x0, 5). PC Adder outputs 0x04. At the next clock edge, InstrD = 0x00500293, PCD = 0x00, PCPlus4D = 0x04.
- Cycle 2: PC advances to 0x04. mem[1] = 0x00300313 (addi x6, x0, 3). PC Adder outputs 0x08.
- Cycle 3: PC = 0x08. mem[2] = 0x006283B3 (add x7, x5, x6). This is an R-type instruction.
- Cycle 4: PC = 0x0C. mem[3] = 0x00002403 (lw x8, 0(x0)). A load word instruction.
- Cycle 5: PC = 0x10. mem[4] = 0x00100493 (addi x9, x0, 1).
- Cycle 6: PC = 0x14. mem[5] = 0x00940533 (add x10, x8, x9).

Verilog Source Code: Fetch_Cycle.v

```
`ifndef FETCH_CYCLE_V
`define FETCH_CYCLE_V

`include "PC.v"
`include "PC_Adder.v"
`include "Mux.v"
`include "Instruction_Memory.v"

module Fetch_Cycle (
    input clk,
    input rst,
    input PCSrcE,
    input [31:0] PCTargetE,
    output reg [31:0] InstrD,
    output reg [31:0] PCD,
    output reg [31:0] PCPlus4D
);

    // Declaration of Interim Wires
    wire [31:0] PC_F, PCF, PCPlus4F, InstrF;

    // PC Multiplexer (Select PC+4 vs branch/jump destination target address)
    Mux PC_Mux (
        .a(PCPlus4F),
        .b(PCTargetE),
        .s(PCSrcE),
        .c(PC_F)
    );

    // Program Counter Register Instantiation
    PC_Module Program_Counter (
        .clk(clk),
        .rst(rst),
        .PC(PCF),
        .PC_Next(PC_F)
    );

    // Instruction Memory Instantiation
    Instruction_Memory IMEM (
        .rst(rst),
        .A(PCF),
        .RD(InstrF)
    );
```

```
44 |
45 |     // PC Adder (Calculates PC + 4)
46 |     PC_Adder PC_adder (
47 |         .a(PCF),
48 |         .b(32'h00000004),
49 |         .c(PCPlus4F)
50 |     );
51 |
52 |     // Sequential Pipeline Register Logic (updates at clock edge)
53 |     always @(posedge clk or negedge rst) begin
54 |         if(rst == 1'b0) begin
55 |             InstrD   <= 32'h00000000;
56 |             PCD      <= 32'h00000000;
57 |             PCPlus4D <= 32'h00000000;
58 |         end
59 |         else begin
60 |             InstrD   <= InstrF;
61 |             PCD      <= PCF;
62 |             PCPlus4D <= PCPlus4F;
63 |         end
64 |     end
65 |
66 | endmodule
67 |
68 | `endif
```

Section 2: Decode Stage (ID) - Overview

The Decode Cycle (ID) is the second stage of the RISC-V pipelined processor. In this stage, the processor interprets the fetched instruction by extracting control signals, reading register operands, and sign-extending the immediate field. Three key sub-modules work together:

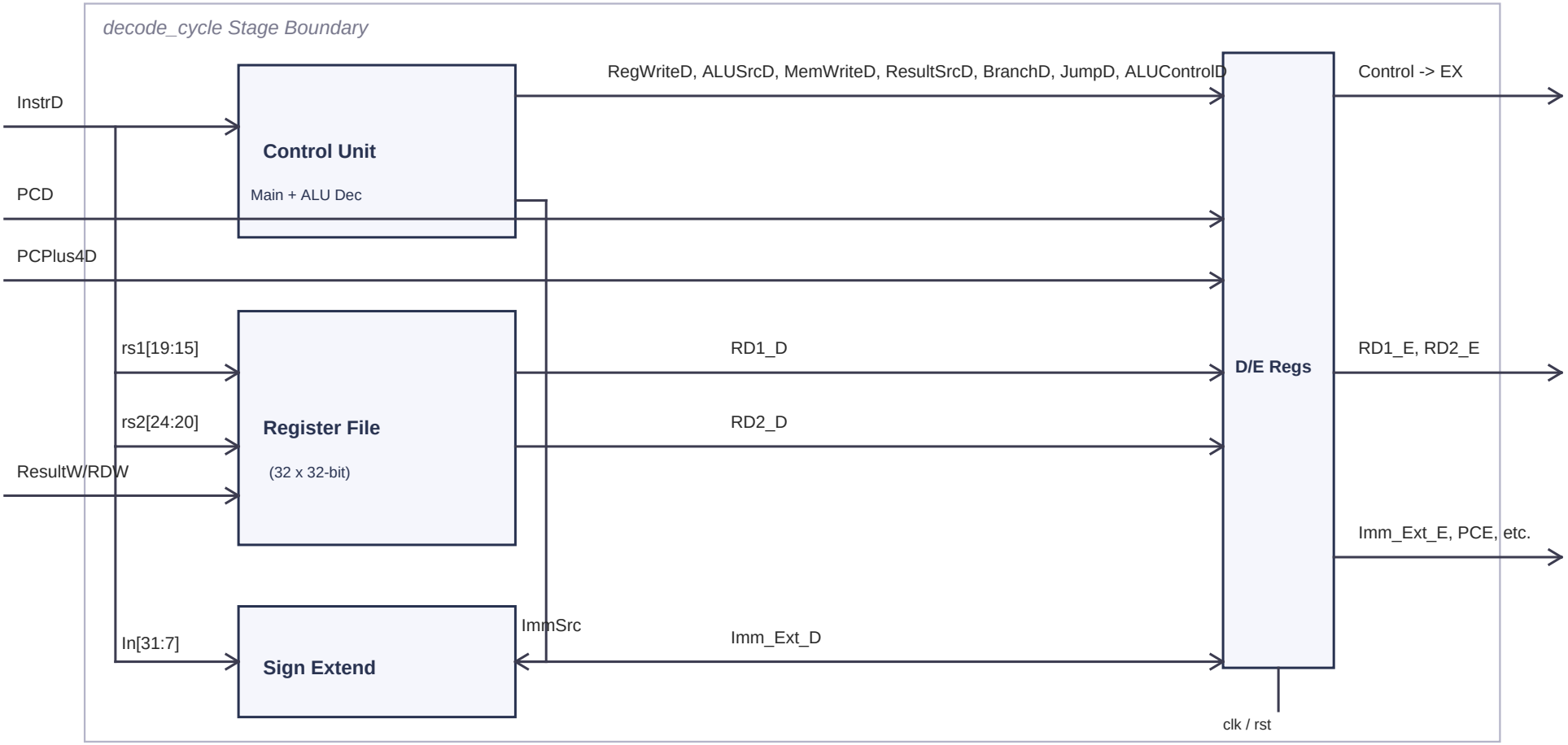
1. Control Unit (Control_Unit_Top): Decodes the 7-bit opcode (and funct3/funct7 fields) into control signals that govern the datapath for the rest of the pipeline (RegWrite, ALUSrc, MemWrite, Branch, Jump, ALUControl, etc.).
2. Register File: A 32x32-bit register bank with dual asynchronous read ports and one synchronous write port (on the falling clock edge). Register x0 is hardwired to zero.
3. Sign Extend: Extracts and sign-extends the immediate field from the instruction based on the instruction type (I-type, S-type, B-type, J-type).

The Decode-to-Execute (D/E) pipeline registers capture all decoded signals (control + data) at the clock edge.

Decode Stage Interface Summary

Signal	Direction	Width	Description
InstrD [31:0]	Input	32	Instruction from Fetch stage (F/D register)
PCD [31:0]	Input	32	PC value from Fetch stage
PCPlus4D [31:0]	Input	32	PC+4 from Fetch stage
RegWriteW	Input	1	Write enable from Writeback stage (for register file)
RDW [4:0]	Input	5	Destination register address from Writeback stage
ResultW [31:0]	Input	32	Write data from Writeback stage
RegWriteE, ALUSrcE, ...	Output	var	Control signals registered into Execute stage (D/E regs)
RD1_E, RD2_E [31:0]	Output	32	Register read data registered into Execute stage
Imm_Ext_E [31:0]	Output	32	Sign-extended immediate registered into Execute stage

Decode Stage Block Diagram



Control Unit Deep Dive: Main Decoder

The Main Decoder takes the 7-bit opcode (InstrD[6:0]) and generates all primary control signals. The ALU Decoder then refines the ALUControl output using ALUOp, funct3, and funct7[5].

Main Decoder Truth Table

Instruction Type	Opcode	RegW	ImmSrc	ALUSrc	MemW	ResSrc	Branch	Jump	ALUOp	Supported Instructions
R-type (ALU)	0110011	1	00	0	0	00	0	0	10	add, sub, and, or, slt
I-type (ALU)	0010011	1	00	1	0	00	0	0	00	addi, andi, ori, slti
I-type (Load)	0000011	1	00	1	0	01	0	0	00	lw
S-type (Store)	0100011	0	01	1	1	00	0	0	00	sw
B-type (Branch)	1100011	0	10	0	0	00	1	0	01	beq
J-type (Jump)	1101111	1	11	0	0	10	0	1	00	jal

ALU Decoder Truth Table

ALUOp	funct3	{Op[5],funct7[5]}	ALUControl[2:0]	Operation	Explanation
00	X (don't care)	X	000	ADD	Load/Store/I-type ALU: always add for address calc
01	X	X	001	SUB	Branch: subtract to compare registers
10	000	11	001	SUB	R-type sub: Op[5]=1 (R-type) and funct7[5]=1
10	000	!= 11	000	ADD	R-type add or I-type addi
10	010	X	101	SLT	Set Less Than
10	110	X	011	OR	Bitwise OR
10	111	X	010	AND	Bitwise AND

Register File & Sign Extend Deep Dives

Register File Architecture

The Register File contains 32 registers (x0 - x31), each 32 bits wide. It has the following ports:
- A1 [4:0], A2 [4:0]: Read address ports (connected to InstrD[19:15] = rs1 and InstrD[24:20] = rs2)
- RD1 [31:0], RD2 [31:0]: Read data outputs (asynchronous combinational reads)
- A3 [4:0]: Write address port (connected to RDW from Writeback stage)
- WD3 [31:0]: Write data port (connected to ResultW from Writeback stage)
- WE3: Write enable (connected to RegWriteW from Writeback stage)

Dual-Read Asynchronous: Both read ports output data combinationally without waiting for a clock edge.
Single-Write on Negedge: The write occurs on the FALLING edge of the clock (negedge clk). This is a critical design choice - by writing on the falling edge, the Writeback stage can write a result in the first half of the cycle, and the Decode stage can read the updated value in the second half, resolving potential RAW (Read After Write) hazards within the same cycle.

x0 Hardwired to Zero: Both read outputs return 0 when A1 or A2 is 5'b00000. The write logic also prevents writing to x0 (A3 != 5'b00000 check).

Sign Extend Unit: Immediate Encoding by Type

ImmSrc	Type	Bit Extraction from Instruction	Immediate Range / Purpose
2'b00	I-type	{20{In[31]}, In[31:20]} -- sign-extended 12-bit	Arithmetic immediates (addi), Load offsets (lw)
2'b01	S-type	{20{In[31]}, In[31:25], In[11:7]} -- split 12-bit	Store offsets (sw): imm split across funct7 and rd fields
2'b10	B-type	{20{In[31]}, In[7], In[30:25], In[11:8], 1'b0}	Branch offsets: 13-bit with LSB=0 (multiples of 2)
2'b11	J-type	{12{In[31]}, In[19:12], In[20], In[30:21], 1'b0}	Jump offsets (jal): 21-bit with LSB=0 (multiples of 2)

D/E Pipeline Registers

The Decode-to-Execute pipeline registers capture ALL control and data signals at posedge clk:
Control: RegWriteE, ALUSrcE, MemWriteE, ResultSrcE[1:0], BranchE, JumpE, ALUControlE[2:0]
Data: RD1_E[31:0], RD2_E[31:0], Imm_Ext_E[31:0], RD_E[4:0] (= InstrD[11:7], the rd field)
PC: PCE[31:0] (= PCD), PCPlus4E[31:0] (= PCPlus4D)

On reset, all signals are cleared to zero. This ensures no spurious writes or branches occur during reset.

Decode Stage: Example Walkthrough -- addi x5, x0, 5 (0x00500293)

Instruction Bit Fields:

```
Hex: 0x00500293
Bin: 0000 0000 0101 0000 0000 0010 1001 0011

  [31:20]  imm     = 000000000101 = 5 (decimal)
  [19:15]  rs1     = 00000 = x0
  [14:12]  funct3  = 000
  [11:7]   rd      = 00101 = x5
  [6:0]    opcode  = 0010011 = I-type ALU (addi)
```

Control Unit Outputs:

Signal	Value	Reason
RegWrite	1	I-type ALU writes result to rd (x5)
ImmSrc	2'b00	I-type immediate encoding
ALUSrc	1	ALU input B uses the immediate (not rs2)
MemWrite	0	Not a store instruction
ResultSrc	2'b00	Result comes from ALU (not memory or PC+4)
Branch	0	Not a branch instruction
Jump	0	Not a jump instruction
ALUOp	2'b00	I-type ALU: default add
ALUControl	3'b000	ADD operation (ALUOp=00 => always ADD)

Register File Reads:

- A1 = InstrD[19:15] = 00000 (x0). RD1_D = 0 (x0 is hardwired to zero).
- A2 = InstrD[24:20] = 00101 (x5). RD2_D = Register[5] (value of x5, initially 0).

Sign Extend Output:

- ImmSrc = 2'b00 (I-type). Imm_Ext_D = {{20{In[31]}}, In[31:20]} = {{20{0}}, 000000000101} = 32'h00000005 = 5.

At the next clock edge, all these values are captured in the D/E pipeline registers.

Verilog Source Code: Control_Unit_Top.v

```
`ifndef CONTROL_UNIT_TOP_V
`define CONTROL_UNIT_TOP_V

module Control_Unit_Top (
    input [6:0] Op,
    input [2:0] funct3,
    input funct7_5,           // Bit 30 of instruction (funct7[5])
    output RegWrite,
    output [1:0] ImmSrc,
    output ALUSrc,
    output MemWrite,
    output [1:0] ResultSrc,   // 2-bit to support Jumps (PC+4)
    output Branch,
    output [2:0] ALUControl,
    output Jump
);

    // Internal wire for ALUOp
    wire [1:0] ALUOp;

    // --- Main Decoder Logic ---
    assign RegWrite = (Op == 7'b0000011 || Op == 7'b0110011 || Op == 7'b0010011 || Op == 7'b1101111) ? 1'b1 : 1'b0;

    assign ImmSrc   = (Op == 7'b0100011) ? 2'b01 :
                      (Op == 7'b1100011) ? 2'b10 :
                      (Op == 7'b1101111) ? 2'b11 : // J-type immediate
                                           2'b00 ;

    assign ALUSrc   = (Op == 7'b0000011 || Op == 7'b0100011 || Op == 7'b0010011) ? 1'b1 : 1'b0;

    assign MemWrite = (Op == 7'b0100011) ? 1'b1 : 1'b0;

    assign ResultSrc = (Op == 7'b0000011) ? 2'b01 : // Memory data
                       (Op == 7'b1101111) ? 2'b10 : // PC + 4 (for Jumps)
                                            2'b00 ; // ALU result

    assign Branch   = (Op == 7'b1100011) ? 1'b1 : 1'b0;

    assign Jump     = (Op == 7'b1101111) ? 1'b1 : 1'b0;

    assign ALUOp    = (Op == 7'b0110011) ? 2'b10 :
                      (Op == 7'b1100011) ? 2'b01 :
                                           2'b00 ;
```

```
44 |
45 |     // --- ALU Decoder Logic ---
46 |     assign ALUControl = (ALUOp == 2'b00) ? 3'b000 : // Add (Load/Store)
47 |                         (ALUOp == 2'b01) ? 3'b001 : // Subtract (Branch)
48 |
49 |                         // ALUOp == 2'b10 (R-type or I-type ALU)
50 |                         ((ALUOp == 2'b10) && (funct3 == 3'b000) && ({Op[5], funct7_5} == 2'b11)) ? 3'b001 : // Subtract
51 |                         ((ALUOp == 2'b10) && (funct3 == 3'b000) && ({Op[5], funct7_5} != 2'b11)) ? 3'b000 : // Add / Addi
52 |                         ((ALUOp == 2'b10) && (funct3 == 3'b010)) ? 3'b101 : // SLT
53 |                         ((ALUOp == 2'b10) && (funct3 == 3'b110)) ? 3'b011 : // OR
54 |                         ((ALUOp == 2'b10) && (funct3 == 3'b111)) ? 3'b010 : // AND
55 |                                                                    3'b000 ; // Default
56 |
57 | endmodule
58 |
59 | `endif
```

Verilog Source Code: Register_File.v

```
 1 | `ifndef REGISTER_FILE_V
 2 | `define REGISTER_FILE_V
 3 |
 4 | module Register_File (
 5 |     input clk,
 6 |     input rst,
 7 |     input WE3,
 8 |     input [4:0] A1,
 9 |     input [4:0] A2,
10 |     input [4:0] A3,
11 |     input [31:0] WD3,
12 |     output [31:0] RD1,
13 |     output [31:0] RD2
14 | );
15 |
16 |     reg [31:0] Register [31:0];
17 |     integer i;
18 |
19 |     // Asynchronous reads
20 |     // Register x0 is hardwired to 0 in RISC-V
21 |     assign RD1 = (rst == 1'b0) ? 32'h00000000 : ((A1 == 5'b00000) ? 32'h00000000 : Register[A1]);
22 |     assign RD2 = (rst == 1'b0) ? 32'h00000000 : ((A2 == 5'b00000) ? 32'h00000000 : Register[A2]);
23 |
24 |     // Synchronous write on falling edge of clock to prevent RAW hazards in same cycle
25 |     always @(negedge clk or negedge rst) begin
26 |         if (rst == 1'b0) begin
27 |             for (i = 0; i < 32; i = i + 1) begin
28 |                 Register[i] <= 32'h00000000;
29 |             end
30 |         end else if (WE3 && (A3 != 5'b00000)) begin
31 |             Register[A3] <= WD3;
32 |         end
33 |     end
34 |
35 | endmodule
36 |
37 | `endif
```

Verilog Source Code: Sign_Extend.v

```
 1 | `ifndef SIGN_EXTEND_V
 2 | `define SIGN_EXTEND_V
 3 |
 4 | module Sign_Extend (
 5 |     input [31:7] In,
 6 |     input [1:0] ImmSrc,
 7 |     output reg [31:0] Imm_Ext
 8 | );
 9 |
10 |     always @(*) begin
11 |         case (ImmSrc)
12 |             // I-Type immediate (Arithmetic immediates, Loads)
13 |             2'b00:   Imm_Ext = { {20{In[31]}}, In[31:20] };
14 |
15 |             // S-Type immediate (Stores)
16 |             2'b01:   Imm_Ext = { {20{In[31]}}, In[31:25], In[11:7] };
17 |
18 |             // B-Type immediate (Branches)
19 |             2'b10:   Imm_Ext = { {20{In[31]}}, In[7], In[30:25], In[11:8], 1'b0 };
20 |
21 |             // J-Type immediate (Jumps - JAL)
22 |             2'b11:   Imm_Ext = { {12{In[31]}}, In[19:12], In[20], In[30:21], 1'b0 };
23 |
24 |             default: Imm_Ext = 32'h00000000;
25 |         endcase
26 |     end
27 |
28 | endmodule
29 |
30 | `endif
```

Verilog Source Code: Decode_Cycle.v

```
`ifndef DECODE_CYCLE_V
`define DECODE_CYCLE_V

`include "Control_Unit_Top.v"
`include "Register_File.v"
`include "Sign_Extend.v"

module decode_cycle (
    input clk,
    input rst,
    input [31:0] InstrD,
    input [31:0] PCD,
    input [31:0] PCPlus4D,
    input RegWriteW,
    input [4:0] RDW,
    input [31:0] ResultW,
    output reg RegWriteE,
    output reg ALUSrcE,
    output reg MemWriteE,
    output reg [1:0] ResultSrcE,   // 2-bit registered output
    output reg BranchE,
    output reg JumpE,              // Registered Jump control output
    output reg [2:0] ALUControlE,
    output reg [31:0] RD1_E,
    output reg [31:0] RD2_E,
    output reg [31:0] Imm_Ext_E,
    output reg [4:0] RD_E,
    output reg [31:0] PCE,
    output reg [31:0] PCPlus4E
);

    // Declare Interim Wires (outputs from the combinational blocks)
    wire RegWriteD, ALUSrcD, MemWriteD, BranchD, JumpD;
    wire [1:0] ResultSrcD;
    wire [1:0] ImmSrcD;
    wire [2:0] ALUControlD;
    wire [31:0] RD1_D, RD2_D, Imm_Ext_D;

    // 1. Control Unit Instantiation
    Control_Unit_Top control (
        .Op(InstrD[6:0]),
        .RegWrite(RegWriteD),
        .ImmSrc(ImmSrcD),
```

```
44 |         .ALUSrc(ALUSrcD),
45 |         .MemWrite(MemWriteD),
46 |         .ResultSrc(ResultSrcD),     // Connected natively
47 |         .Branch(BranchD),
48 |         .funct3(InstrD[14:12]),
49 |         .funct7_5(InstrD[30]),       // Connected to bit 30
50 |         .ALUControl(ALUControlD),
51 |         .Jump(JumpD)                // Connected natively
52 |     );
53 |
54 |     // 2. Register File Instantiation
55 |     Register_File reg_file (
56 |         .clk(clk),
57 |         .rst(rst),
58 |         .WE3(RegWriteW),
59 |         .A1(InstrD[19:15]),
60 |         .A2(InstrD[24:20]),
61 |         .A3(RDW),
62 |         .WD3(ResultW),
63 |         .RD1(RD1_D),
64 |         .RD2(RD2_D)
65 |     );
66 |
67 |     // 3. Sign Extension Instantiation
68 |     Sign_Extend extension (
69 |         .In(InstrD[31:7]),           // Passing the 25-bit slice matching Sign_Extend.v port
70 |         .Imm_Ext(Imm_Ext_D),
71 |         .ImmSrc(ImmSrcD)
72 |     );
73 |
74 |     // Declaring Register Logic (updates at posedge clk directly to output registers)
75 |     always @(posedge clk or negedge rst) begin
76 |         if (rst == 1'b0) begin
77 |             RegWriteE   <= 1'b0;
78 |             ALUSrcE     <= 1'b0;
79 |             MemWriteE   <= 1'b0;
80 |             ResultSrcE  <= 2'b00;
81 |             BranchE     <= 1'b0;
82 |             JumpE       <= 1'b0;
83 |             ALUControlE <= 3'b000;
84 |             RD1_E       <= 32'h00000000;
85 |             RD2_E       <= 32'h00000000;
86 |             Imm_Ext_E   <= 32'h00000000;
87 |             RD_E        <= 5'h00;
88 |             PCE         <= 32'h00000000;
89 |             PCPlus4E    <= 32'h00000000;
```

```
 90 |         end
 91 |         else begin
 92 |             RegWriteE   <= RegWriteD;
 93 |             ALUSrcE     <= ALUSrcD;
 94 |             MemWriteE   <= MemWriteD;
 95 |             ResultSrcE  <= ResultSrcD;
 96 |             BranchE     <= BranchD;
 97 |             JumpE       <= JumpD;
 98 |             ALUControlE <= ALUControlD;
 99 |             RD1_E       <= RD1_D;
100 |             RD2_E       <= RD2_D;
101 |             Imm_Ext_E   <= Imm_Ext_D;
102 |             RD_E        <= InstrD[11:7]; // rd register address
103 |             PCE         <= PCD;
104 |             PCPlus4E    <= PCPlus4D;
105 |         end
106 |     end
107 |
108 | endmodule
109 |
110 | `endif
```

Section 3: Execute Stage (EX) - Overview

The Execute Cycle (EX) is the third stage of the RISC-V pipelined processor. In this stage, the processor performs actual mathematical computations and address additions. To build a working execute stage, five key modules are integrated together:

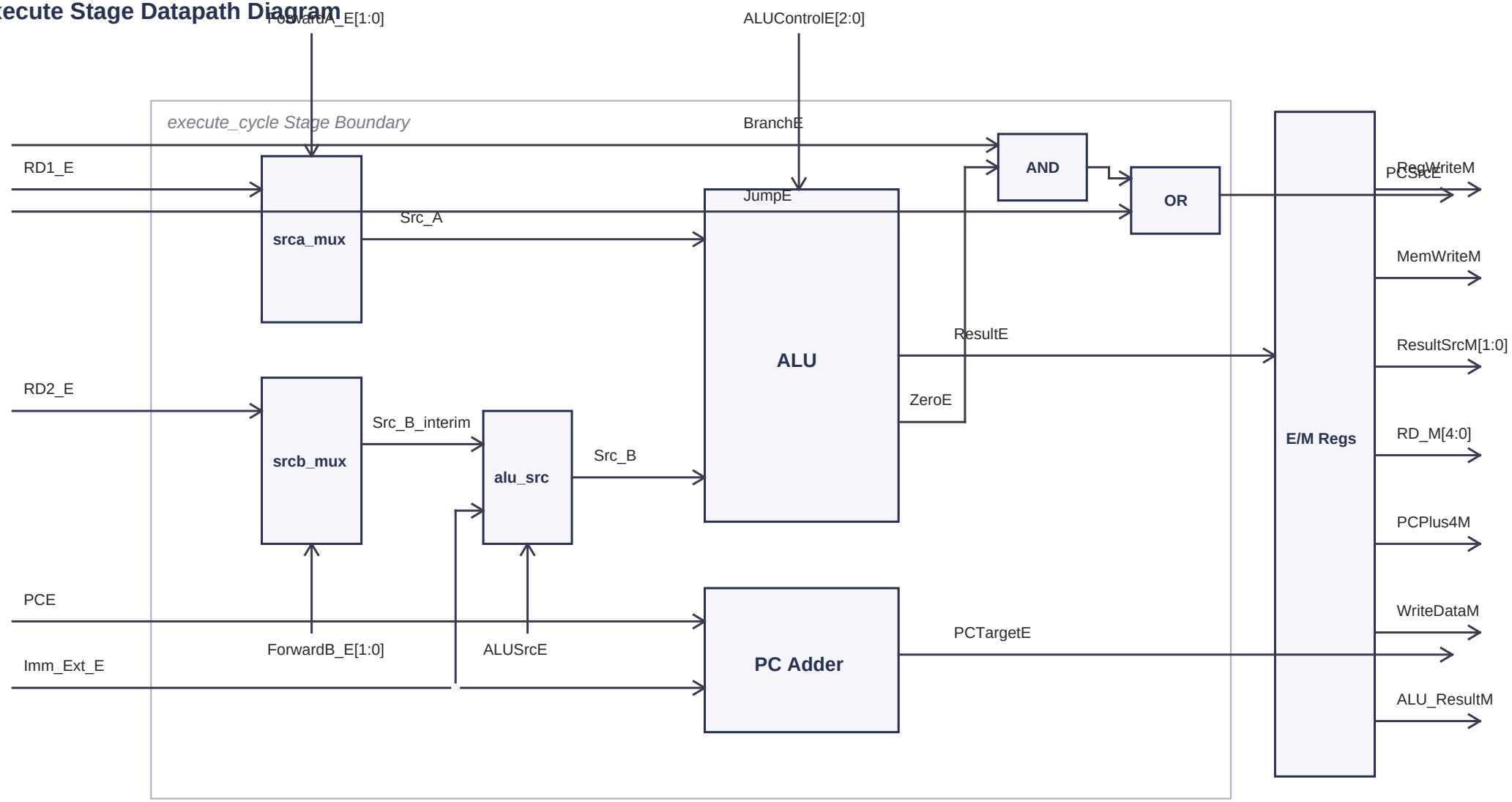
1. ALU (Arithmetic Logic Unit): Executes core arithmetic, logical, and comparison operations.
2. PC Target Adder: Adds the current PC value to the sign-extended immediate to calculate the target branch address.
3. Forwarding Multiplexers: Select register operands from Decode, or forward values from Memory/Writeback to resolve dependencies.
4. PC Source Select Gate: Combinational logic (AND & OR gates) that decides if the next PC is sequential or a branch/jump target.
5. Execute-to-Memory (E/M) Registers: Holds the stage results and carries control signals forward to the Memory stage.

This guide explains each block visually and provides the complete Verilog module schematic.

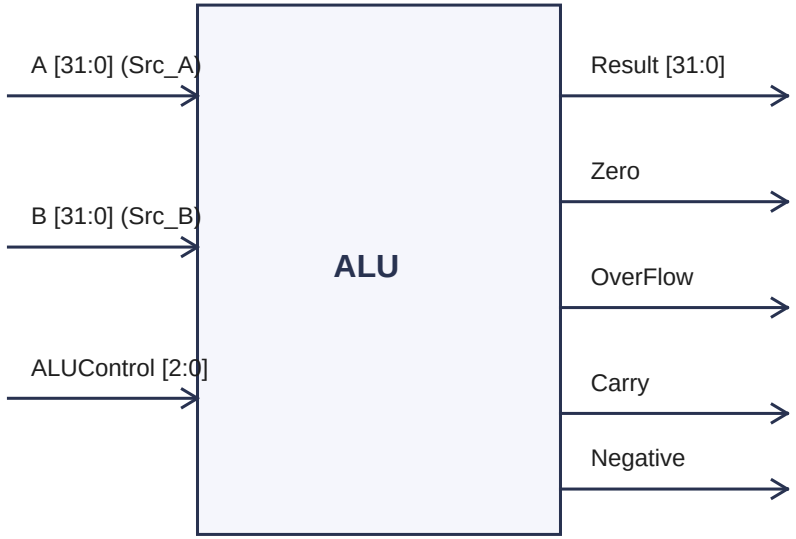
Execute Stage Interface Summary

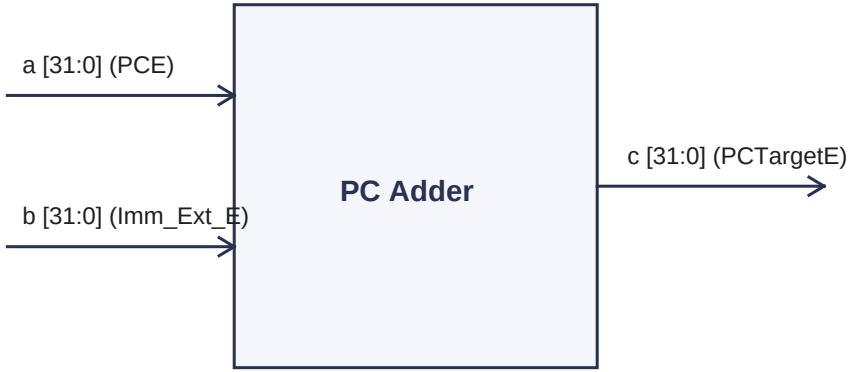
Signal Category	Inputs (EX Stage)	Outputs (EX Stage)
Datapath Paths	RD1_E, RD2_E, PCE, Imm_Ext_E, PCPlus4E, ResultW	PCTargetE, WriteDataM, ALU_ResultM, PCPlus4M
Control Signals	RegWriteE, MemWriteE, ResultSrcE, BranchE, JumpE, ALUSrcE, ALUControlE	RegWriteM, MemWriteM, ResultSrcM, PCSrcE
Hazards & Forwarding	ForwardA_E[1:0], ForwardB_E[1:0] (from Hazard Unit)	-

Execute Stage Datapath Diagram

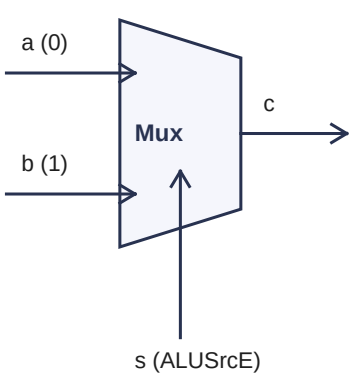


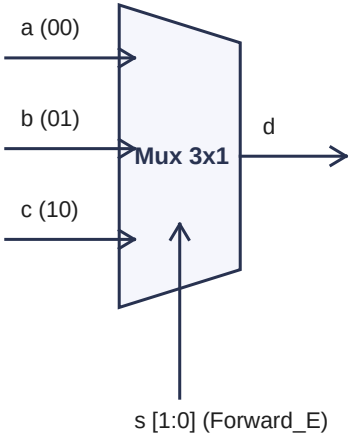
Module Schematics: ALU & PC Target Adder

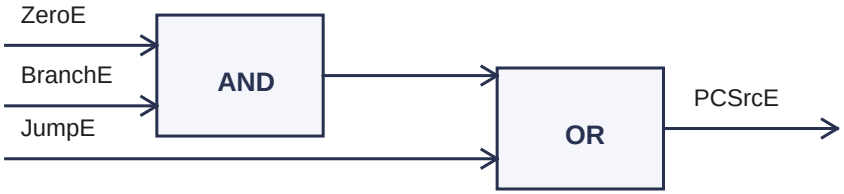




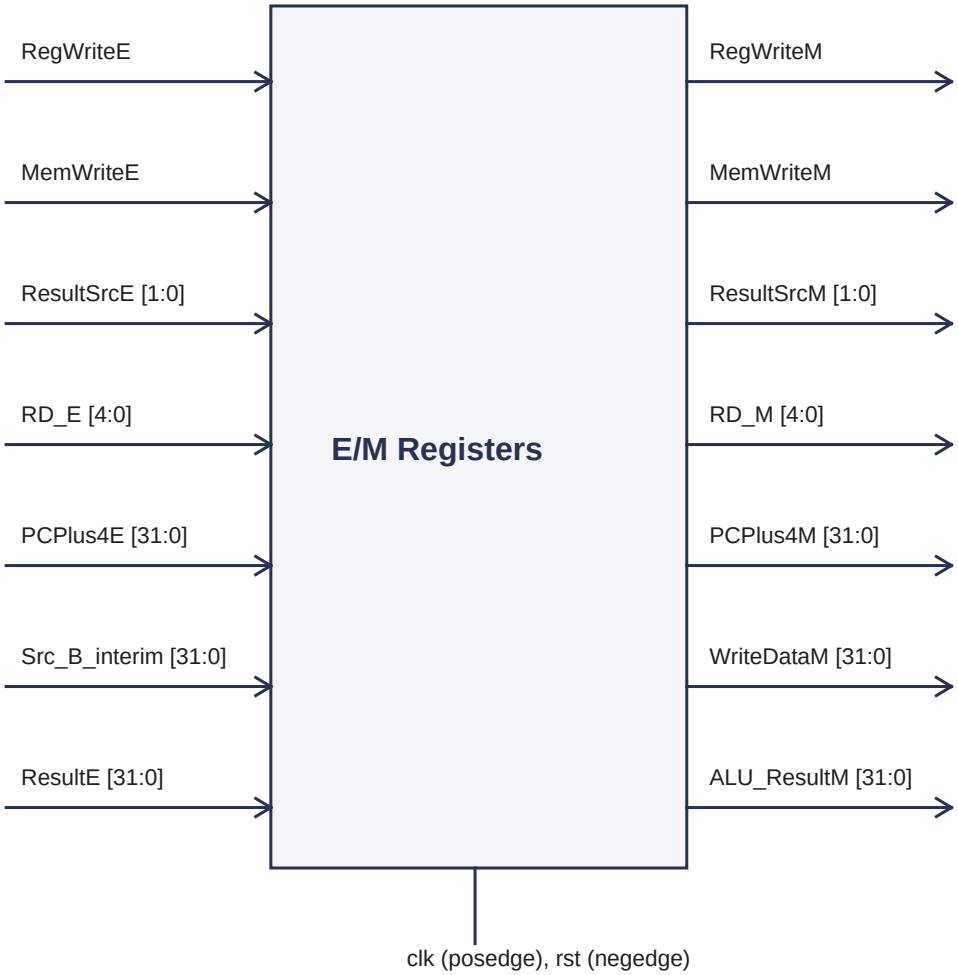
Module Schematics: Multiplexers & PC Source Logic







Module Schematics: Execute-to-Memory Registers (E/M)



Deep Dive: ALU Inputs & Memory Routing (SrcAE, SrcBE, WriteDataE)

ALU Input Signals Mapped

Signal Name	Source in Hardware	What it Contains	Role in CPU Calculation
SrcAE (Source A)	RD1_E (or forwarded ALU_ResultM / ResultW)	32-bit register value from drawer 1 (rs1)	First number fed into the ALU for computation
SrcBE (Source B)	Output of the 2-to-1 ALU Source Mux	RD2_E (or forwarded value) OR Imm_Ext_E constant	Second number fed into the ALU for computation
WriteDataE	Bypassed wire split off directly from RD2_E	The cargo data value we want to write to RAM	Passes data around the ALU to the E/M registers

Key Architecture Concepts Explained:

- Are SrcAE and RD1E the same wire?

* In a basic CPU schematic without forwarding, they are exactly the same physical wire.
* In a fully pipelined CPU, they are NOT always the same. If a data hazard occurs (a previous instruction is writing to our source register), the forwarding multiplexer (srca_mux) will override RD1_E and inject the corrected value from the Memory or Writeback stage directly into SrcAE. This prevents the CPU from having to stall.

- Why does the RD2E wire split into WriteDataE?

* For store instructions (sw), we must calculate the address (e.g. x6 + 4) and carry the data value to write (e.g. x7).
* The ALU needs the base address x6 (SrcAE) and the immediate offset 4 (SrcBE). Since the ALU Source Mux selects the immediate offset (4), we must split the register data (x7) BEFORE it reaches the multiplexer.
* This split wire is labeled WriteDataE (or cargo). It travels around the ALU directly to the pipeline registers so it can be written to memory during the next stage.

Case Study: Memory Addressing and Load (lw) vs Store (sw) Datapath

How Registers Store Memory Addresses

To the Register File, every value is just a 32-bit number. A number like 0x1000 is no different from the count of apples (50).
The CPU treats a number as an address only because of how it is routed. When a programmer wants to load or store data, they first load the address (e.g. 0x1000) into a base register (e.g. x6). During execution, this number is read into RD1_E. The ALU adds the offset to compute the final address, which is then fed into the Address port of the RAM chip. Because it enters the RAM's Address port, the RAM chip treats the number as a physical drawer coordinate in memory.

Datapath Flow Comparison

Signal / Port	Load Instruction: lw x5, 4(x6)	Store Instruction: sw x7, 4(x6)
RD1_E (rs1)	Base address from x6 (e.g. 0x1000)	Base address from x6 (e.g. 0x1000)
RD2_E (rs2)	Ignored (not used during load)	Cargo data value to be written from x7 (e.g. 99)
ALUSrcE (Mux Switch)	1 (Selects Imm_Ext_E offset)	1 (Selects Imm_Ext_E offset)
SrcAE (ALU Input A)	0x1000 (Base Address)	0x1000 (Base Address)
SrcBE (ALU Input B)	4 (Offset value)	4 (Offset value)
ALU Result (Address)	0x1004 (Target Address to read from RAM)	0x1004 (Target Address to write into RAM)
WriteDataM (Cargo)	N/A (Ignored)	99 (Data value to store at address 0x1004)

Deep Dive: Program Counter, Branch/Jump Logic and Target Adder

PCE, Adder & Routing Architecture Summary

Component / Signal	Key Concept	Hardware Routing & Connection Details
PCE (Program Counter)	Active instruction address	Passed from PCD at clock edge. Wires directly to the top input of the target adder.
PC Target Adder (+ block)	Parallel Target Address Calculator	Adds PCE and Imm_Ext_E in parallel to ALU. Simple combinational design (no clk/rst).
PCTargetE (Output)	Computed jump address	Outputs sum PCE + Imm_Ext_E. Routes back to Fetch stage to load next target PC.

PC Source Select Logic (AND / OR Gates)

The CPU decides whether to load the sequential PC + 4 or redirect to the branch target PCTargetE using the control equation: PCSrcE = (ZeroE & BranchE) | JumpE. This decision is resolved by two logic gates working together:

1. The AND Gate (Conditional Branches): Takes inputs BranchE (tells the CPU if this instruction is a conditional branch) and ZeroE (comes from the ALU, high only if the comparison matched). The AND gate output goes high only if BOTH are 1. This prevents the CPU from jumping during arithmetic instructions where the ALU result happens to be zero, as BranchE is 0.
2. The OR Gate (Unconditional Jump Override): Takes the output of the AND gate and JumpE (tells the CPU if this instruction is an unconditional jump, like jal). Since unconditional jumps must always branch regardless of data comparisons, JumpE = 1 bypasses the AND gate check, pulling PCSrcE high instantly.

BranchE (Ctrl)	ZeroE (ALU)	JumpE (Ctrl)	PCSrcE (Select)	Control Decision and Description
0	Any	0	0	Sequential Execution: Fetch stage loads normal PC + 4 (no branch/jump).
1	0	0	0	Branch Failed: Condition was not met. Fetch stage loads sequential PC + 4.
1	1	0	1	Branch Succeeded: Condition met! Fetch stage redirects to PCTargetE.
Any	Any	1	1	Unconditional Jump: Jump instruction active. Always redirect to PCTargetE.

Deep Dive: ALU Operations, Codes and Two's Complement Addition/Subtraction

ALU Control Truth Table

ALUControl[2:0]	Operation Name	Mathematical Formula	Hardware Implementation Details
3'b000	ADD	Result = A + B	Adds the 32-bit values of A and B using the parallel adder core.
3'b001	SUB	Result = A - B	Computes addition of A and the Two's Complement of B: A + (~B + 1).
3'b010	AND	Result = A & B	Performs a bit-by-bit logical AND gate comparison of inputs A and B.
3'b011	OR	Result = A | B	Performs a bit-by-bit logical OR gate comparison of inputs A and B.
3'b101	SLT (Set Less Than)	Result = (A < B) ? 1 : 0	Subtracts B from A. If negative, Sum[31] = 1, outputting 32'h00000001.

Two's Complement Arithmetic Dual-Use Design:

How does the ALU do addition and subtraction in a single equation?

* Mathematically, subtraction is equivalent to adding a negative number: A - B = A + (-B).
* In signed binary hardware, we represent negative numbers using Two's Complement: -B = (~B) + 1 (invert all bits and add 1).
* Thus: A - B = A + (~B + 1).
* By looking at the control signals, we notice that subtraction commands (SUB 3'b001, SLT 3'b101) both have a 1 in their least significant bit (ALUControl[0] == 1). Addition commands (ADD 3'b000) have a 0 in their LSB.
* We use this bit as the trigger wire: Sum = (ALUControl[0] == 0) ? (A + B) : (A + (~B + 1)). This allows a single addition circuit to perform subtraction seamlessly, saving valuable physical chip space.

Deep Dive: ALU Status Flag Logic Derivation and Summary Table

Signed Overflow (V) Logic Derivation

Signed overflow occurs when the math result has a sign that is physically impossible.

* Adding two positive numbers must yield a positive result. If it yields a negative result (Sum[31]=1), overflow occurred.
* Adding two negative numbers must yield a negative result. If it yields a positive result (Sum[31]=0), overflow occurred.
* Adding opposite signs can never overflow.

Sign A (Sa)	Effective Sign B (S_eff)	Expected Sign Sum	Actual Sign Sum (Ss)	Overflow Flag (V)
0 (Positive)	0 (Positive)	0 (Positive)	0 (Positive)	0
0 (Positive)	0 (Positive)	0 (Positive)	1 (Negative)	1 (Overflow)
0 (Positive)	1 (Negative)	Any	Any	0 (Impossible)
1 (Negative)	0 (Positive)	Any	Any	0 (Impossible)
1 (Negative)	1 (Negative)	1 (Negative)	0 (Positive)	1 (Overflow)

Flag Name	Math Criterion / Condition	Logic Derivation & Sign Checks	Verilog Code implementation
OverFlow (V)	Mathematically impossible sign results	Sa == S_eff and Sa != S_sum. Active in ADD/SUB	assign OverFlow = ((Sum[31]^A[31]) & ~(ALUControl[0]^B[31]^A[31]) & ~ALUControl[1]);
Carry (C)	Unsigned overflow or borrow	Captures the 33rd bit (Cout) of addition/subtraction	assign Carry = (~ALUControl[1] & Cout);
Negative (N)	Result is less than zero	Checks MSB (sign bit) of 32-bit result	assign Negative = Result[31];
Zero (Z)	Result is exactly zero	Checks if all bits are 0 using reduction AND (&)	assign Zero = &(~Result);

Verilog Source Code: Execute_Cycle.v Reference

```
`ifndef EXECUTE_CYCLE_V
`define EXECUTE_CYCLE_V

`include "Mux.v"
`include "ALU.v"
`include "PC_Adder.v"

module execute_cycle (
    input clk,
    input rst,
    input RegWriteE,
    input ALUSrcE,
    input MemWriteE,
    input [1:0] ResultSrcE,   // 2-bit input to support Jumps (PC+4)
    input BranchE,
    input JumpE,              // JumpE input
    input [2:0] ALUControlE,
    input [31:0] RD1_E,
    input [31:0] RD2_E,
    input [31:0] Imm_Ext_E,
    input [4:0] RD_E,
    input [31:0] PCE,
    input [31:0] PCPlus4E,
    input [31:0] ResultW,
    input [1:0] ForwardA_E,
    input [1:0] ForwardB_E,
    output PCSrcE,            // PC Mux selector output (Combinational)
    output [31:0] PCTargetE,  // Calculated jump/branch destination address (Combinational)
    output reg RegWriteM,
    output reg MemWriteM,
    output reg [1:0] ResultSrcM,  // 2-bit output
    output reg [4:0] RD_M,
    output reg [31:0] PCPlus4M,
    output reg [31:0] WriteDataM,
    output reg [31:0] ALU_ResultM
);

    // Declaration of Interim Wires
    wire [31:0] Src_A, Src_B_interim, Src_B;
    wire [31:0] ResultE;
    wire ZeroE;

    // 3-by-1 Mux for Source A (ALU input A selection with forwarding)
```

```
44 |     Mux_3_by_1 srca_mux (
45 |         .a(RD1_E),
46 |         .b(ResultW),
47 |         .c(ALU_ResultM),
48 |         .s(ForwardA_E),
49 |         .d(Src_A)
50 |     );
51 |
52 |     // 3-by-1 Mux for Source B (ALU input B selection with forwarding)
53 |     Mux_3_by_1 srcb_mux (
54 |         .a(RD2_E),
55 |         .b(ResultW),
56 |         .c(ALU_ResultM),
57 |         .s(ForwardB_E),
58 |         .d(Src_B_interim)
59 |     );
60 |
61 |     // ALU Source Multiplexer (Select register operand vs immediate constant)
62 |     Mux alu_src_mux (
63 |         .a(Src_B_interim),
64 |         .b(Imm_Ext_E),
65 |         .s(ALUSrcE),
66 |         .c(Src_B)
67 |     );
68 |
69 |     // Arithmetic Logic Unit (ALU) Instantiation
70 |     ALU alu (
71 |         .A(Src_A),
72 |         .B(Src_B),
73 |         .Result(ResultE),
74 |         .ALUControl(ALUControlE),
75 |         .OverFlow(),
76 |         .Carry(),
77 |         .Zero(ZeroE),
78 |         .Negative()
79 |     );
80 |
81 |     // Branch Target Address Adder
82 |     PC_Adder branch_adder (
83 |         .a(PCE),
84 |         .b(Imm_Ext_E),
85 |         .c(PCTargetE)
86 |     );
87 |
88 |     // Sequential Pipeline Register Logic (updates at clock edge directly to output registers)
89 |     always @(posedge clk or negedge rst) begin
```

```
 90 |         if(rst == 1'b0) begin
 91 |             RegWriteM  <= 1'b0;
 92 |             MemWriteM  <= 1'b0;
 93 |             ResultSrcM <= 2'b00;
 94 |             RD_M       <= 5'h00;
 95 |             PCPlus4M   <= 32'h00000000;
 96 |             WriteDataM <= 32'h00000000;
 97 |             ALU_ResultM<= 32'h00000000;
 98 |         end
 99 |         else begin
100 |             RegWriteM  <= RegWriteE;
101 |             MemWriteM  <= MemWriteE;
102 |             ResultSrcM <= ResultSrcE;
103 |             RD_M       <= RD_E;
104 |             PCPlus4M   <= PCPlus4E;
105 |             WriteDataM <= Src_B_interim;
106 |             ALU_ResultM<= ResultE;
107 |         end
108 |     end
109 |
110 |     // Combinational Output Assignments
111 |     assign PCSrcE = (ZeroE & BranchE) | JumpE;
112 |
113 | endmodule
114 |
115 | `endif
```

Section 4: Memory Stage (MEM) - Overview

The Memory Cycle (MEM) is the fourth stage of the RISC-V pipelined processor. In this stage, the processor accesses the Data Memory for load and store instructions. For all other instructions, the memory stage simply passes data through to the next stage. The Memory stage contains two key components:

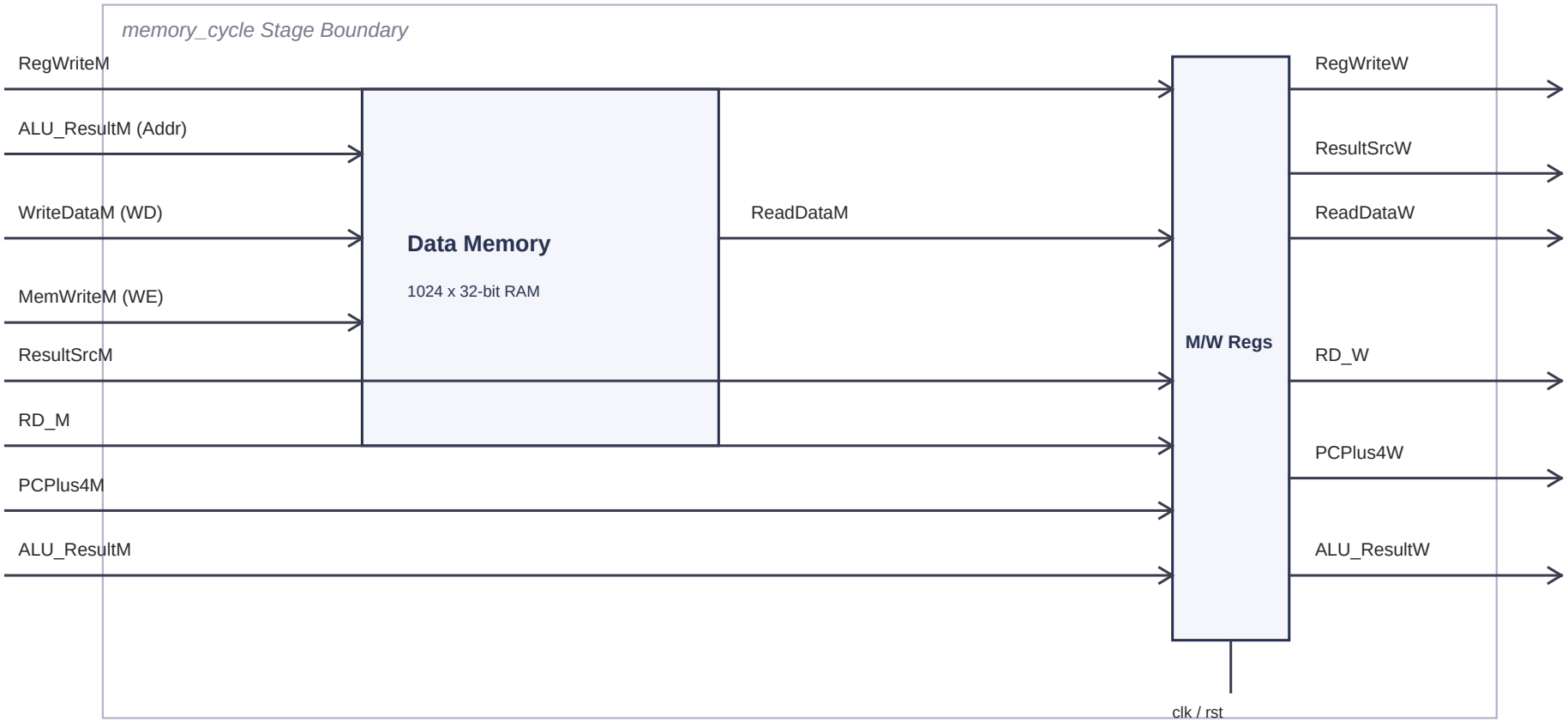
1. Data Memory (DMEM): A 1024-word x 32-bit RAM with synchronous write and asynchronous read. The ALU result from the Execute stage provides the memory address, and WriteDataM provides the data to store (for store instructions). The read data (ReadDataM) is available combinationally.
2. Memory-to-Writeback (M/W) Pipeline Registers: Captures all signals needed by the Writeback stage at the clock edge: RegWriteW, ResultSrcW, RD_W, PCPlus4W, ALU_ResultW, and ReadDataW.

For non-memory instructions (add, addi, etc.), MemWriteM = 0 so no write occurs, and ReadDataM is ignored by the Writeback stage (ResultSrcW selects ALU result instead).

Memory Stage Interface Summary

Signal	Direction	Width	Description
RegWriteM	Input	1	Register write enable (passed through to Writeback)
MemWriteM	Input	1	Data Memory write enable (1 for store instructions)
ResultSrcM [1:0]	Input	2	Result source select (passed through to Writeback)
RD_M [4:0]	Input	5	Destination register address (passed through)
PCPlus4M [31:0]	Input	32	PC+4 (passed through for jal writeback)
WriteDataM [31:0]	Input	32	Data to write to memory (from rs2 via Execute)
ALU_ResultM [31:0]	Input	32	ALU result = memory address for loads/stores
RegWriteW	Output	1	Register write enable for Writeback stage
ResultSrcW [1:0]	Output	2	Result source select for Writeback stage
RD_W [4:0]	Output	5	Destination register for Writeback stage
ALU_ResultW [31:0]	Output	32	ALU result registered for Writeback
ReadDataW [31:0]	Output	32	Data read from memory, registered for Writeback

Memory Stage Block Diagram



Data Memory Deep Dive

Word-Aligned Addressing (A[31:2])

Like the Instruction Memory, the Data Memory uses word-aligned addressing via A[31:2]. The 2 least significant bits of the address are discarded because each 32-bit word occupies 4 consecutive byte addresses. This means:

- Address 0x00 maps to mem[0]
- Address 0x04 maps to mem[1]
- Address 0x08 maps to mem[2]
- Address 0x1000 maps to mem[1024]

The actual index into the memory array is computed as A >> 2 (or equivalently A[31:2]).

Synchronous Write, Asynchronous Read

- Write (Synchronous): On posedge clk, if WE (MemWriteM) is high, the data WD (WriteDataM) is written to mem[A[31:2]]. This ensures writes happen cleanly at the clock edge, preventing race conditions.
- Read (Asynchronous): The output RD = mem[A[31:2]] is available combinationally (no clock needed). On reset (rst == 0), the output is forced to 32'd0. The asynchronous read allows the loaded data to be available within the same cycle for capture by the M/W pipeline register at the next clock edge.

Memory Capacity Calculation

The memory is declared as: reg [31:0] mem [0:1023];

- 1024 entries (indices 0 to 1023)
- Each entry is 32 bits = 4 bytes
- Total capacity: 1024 x 4 = 4096 bytes = 4 KB
- Addressable range: 0x000 to 0xFFC (byte addresses), or mem[0] to mem[1023]

Reset Behavior

The Data Memory initializes mem[0] = 32'h00000000 in an initial block. On reset (rst == 0), the read output is forced to zero, but the memory contents themselves are not cleared (only the initial block sets mem[0]). This is typical for simulation; in real hardware, RAM contents are undefined at power-on.

M/W Pipeline Register Explanation

The M/W pipeline registers capture six signals at posedge clk (or negedge rst for async reset):

- RegWriteW <= RegWriteM (register write enable)
- ResultSrcW <= ResultSrcM (result source selection)
- RD_W <= RD_M (destination register address)
- PCPlus4W <= PCPlus4M (PC+4 for jal writeback)

- ALU_ResultW <= ALU_ResultM (ALU computation result)
- ReadDataW <= ReadDataM (data loaded from memory)

On reset, all outputs are cleared to zero.

Memory Stage: Example Walkthroughs

Example 1: lw x8, 0(x0) -- Load Word

This instruction loads a word from memory address (x0 + 0) = 0x00 into register x8.

Execute Stage Output (inputs to Memory stage):
- ALU_ResultM = 0x00000000 (x0 + immediate 0 = 0x00)
- MemWriteM = 0 (this is a LOAD, not a store -- no write to memory)
- WriteDataM = don't care (ignored since MemWriteM = 0)
- RegWriteM = 1 (we will write the loaded data to x8)
- ResultSrcM = 2'b01 (select memory read data for writeback)
- RD_M = 5'd8 (destination register x8)

Memory Stage Operation:
- Data Memory receives address A = 0x00000000. Index = A[31:2] = 0.
- Asynchronous read: ReadDataM = mem[0] = 0x00000000 (or whatever was stored at mem[0]).
- No write occurs (WE = MemWriteM = 0).
- At posedge clk, M/W registers capture: ReadDataW = ReadDataM, ALU_ResultW = 0x00000000, etc.

Example 2: sw x7, 4(x6) -- Store Word

This instruction stores the value of register x7 into memory address (x6 + 4).
Assume x6 = 0x00000000, x7 = 8 (result of add x7, x5, x6 where x5=5, x6=3).

Execute Stage Output (inputs to Memory stage):
- ALU_ResultM = 0x00000004 (x6 + offset 4 = 0x04)
- MemWriteM = 1 (this is a STORE -- write to memory)
- WriteDataM = 0x00000008 (value of x7 = 8, carried through WriteDataE)
- RegWriteM = 0 (stores do not write to the register file)
- ResultSrcM = 2'b00 (don't care, since RegWriteM = 0)

Memory Stage Operation:
- Data Memory receives address A = 0x00000004. Index = A[31:2] = 1.
- WE = MemWriteM = 1, so at posedge clk: mem[1] <= WriteDataM = 0x00000008.
- The value 8 is now stored at memory address 0x04 (mem[1]).
- ReadDataM is read but ignored since ResultSrcW will not select it (RegWriteM = 0).

Verilog Source Code: Data_Memory.v

```
 1 | `ifndef DATA_MEMORY_V
 2 | `define DATA_MEMORY_V
 3 |
 4 | module Data_Memory (
 5 |     input clk,
 6 |     input rst,
 7 |     input WE,
 8 |     input [31:0] A,
 9 |     input [31:0] WD,
10 |     output [31:0] RD
11 | );
12 |
13 |     // Memory array declaration (1024 words of 32 bits each, ascending indices)
14 |     reg [31:0] mem [0:1023];
15 |
16 |     // Synchronous write logic (using word-aligned indexing A[31:2])
17 |     always @(posedge clk) begin
18 |         if (WE) begin
19 |             mem[A[31:2]] <= WD;
20 |         end
21 |     end
22 |
23 |     // Asynchronous read logic (using word-aligned indexing A[31:2])
24 |     assign RD = (rst == 1'b0) ? 32'd0 : mem[A[31:2]];
25 |
26 |     // Memory initialization
27 |     initial begin
28 |         mem[0] = 32'h00000000;
29 |     end
30 |
31 | endmodule
32 |
33 | `endif
```

Verilog Source Code: Memory_Cycle.v

```
 1 | `ifndef MEMORY_CYCLE_V
 2 | `define MEMORY_CYCLE_V
 3 |
 4 | `include "Data_Memory.v"
 5 |
 6 | module memory_cycle (
 7 |     input clk,
 8 |     input rst,
 9 |     input RegWriteM,
10 |     input MemWriteM,
11 |     input [1:0] ResultSrcM,
12 |     input [4:0] RD_M,
13 |     input [31:0] PCPlus4M,
14 |     input [31:0] WriteDataM,
15 |     input [31:0] ALU_ResultM,
16 |     output reg RegWriteW,
17 |     output reg [1:0] ResultSrcW,
18 |     output reg [4:0] RD_W,
19 |     output reg [31:0] PCPlus4W,
20 |     output reg [31:0] ALU_ResultW,
21 |     output reg [31:0] ReadDataW
22 | );
23 |
24 |     // Interim wire for Data Memory output
25 |     wire [31:0] ReadDataM;
26 |
27 |     // Instantiation of Data Memory Module
28 |     Data_Memory dmem (
29 |         .clk(clk),
30 |         .rst(rst),
31 |         .WE(MemWriteM),
32 |         .WD(WriteDataM),
33 |         .A(ALU_ResultM),
34 |         .RD(ReadDataM)
35 |     );
36 |
37 |     // Memory-to-Writeback Pipeline Registers (M/W Register)
38 |     always @(posedge clk or negedge rst) begin
39 |         if (rst == 1'b0) begin
40 |             RegWriteW   <= 1'b0;
41 |             ResultSrcW  <= 2'b00;
42 |             RD_W        <= 5'h00;
43 |             PCPlus4W    <= 32'h00000000;
```

```
            ALU_ResultW <= 32'h00000000;
            ReadDataW   <= 32'h00000000;
        end
        else begin
            RegWriteW   <= RegWriteM;
            ResultSrcW  <= ResultSrcM;
            RD_W        <= RD_M;
            PCPlus4W    <= PCPlus4M;
            ALU_ResultW <= ALU_ResultM;
            ReadDataW   <= ReadDataM;
        end
    end

endmodule

`endif
```

Section 5: Writeback Stage (WB) - Overview

The Writeback Cycle (WB) is the fifth and final stage of the RISC-V pipelined processor. Its purpose is to select the correct result value and write it back to the register file in the Decode stage. The Writeback stage contains a single key component:

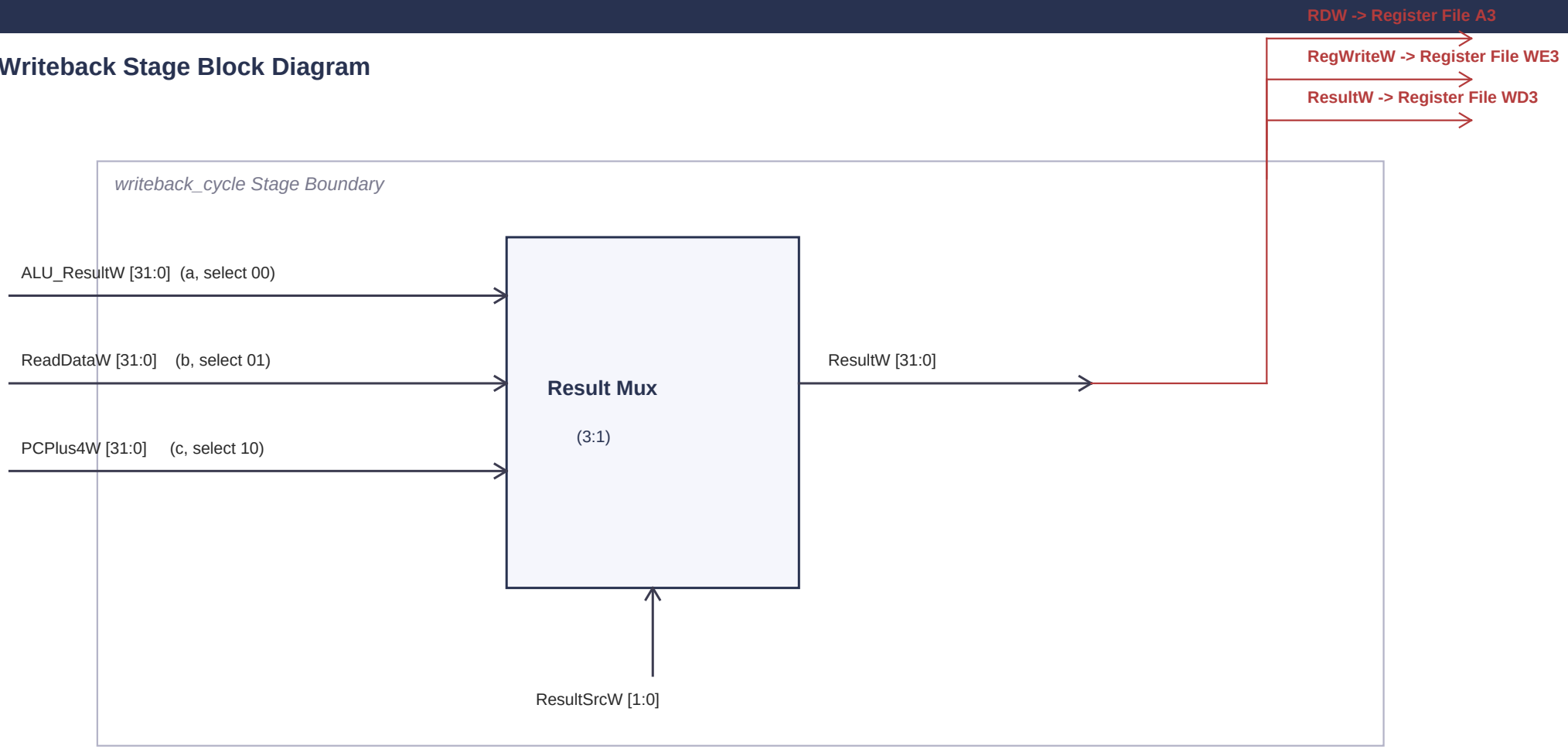
1. 3:1 Result Multiplexer (Mux_3_by_1): Selects between three possible result sources based on ResultSrcW[1:0]: the ALU result, data loaded from memory, or PC+4 (for jal link address).

The selected result (ResultW) is fed back to the Register File's write port (WD3) in the Decode stage, along with the destination register address (RDW = RD_W) and write enable (RegWriteW). If RegWriteW = 1, the register file writes ResultW to register RDW on the falling clock edge.

Writeback Stage Interface Summary

Signal	Direction	Width	Description
ResultSrcW [1:0]	Input	2	Mux select: 00=ALU, 01=MemData, 10=PC+4
ALU_ResultW [31:0]	Input	32	ALU computation result from Memory stage
ReadDataW [31:0]	Input	32	Data loaded from memory (for lw instructions)
PCPlus4W [31:0]	Input	32	PC+4 for jal link address writeback
ResultW [31:0]	Output	32	Selected result fed back to Register File (WD3)
RegWriteW	Feedback	1	Write enable for Register File (from M/W reg)
RD_W / RDW [4:0]	Feedback	5	Destination register address (from M/W reg)

Writeback Stage Block Diagram



ResultSrcW Selection and Feedback Path Details

ResultSrcW Selection Table

ResultSrcW	Mux Input	Selected Signal	When Used	Example Instruction
2'b00	a	ALU_ResultW	R-type, I-type ALU ops	add x7, x5, x6 -> ResultW = 8
2'b01	b	ReadDataW	Load instructions	lw x8, 0(x0) -> ResultW = mem[0]
2'b10	c	PCPlus4W	Jump and link	jal x1, target -> ResultW = PC+4

Feedback Path Explanation

The Writeback stage provides three feedback signals back to the Decode stage's Register File:

1. ResultW [31:0] -> WD3 (Write Data): The selected result value to be written to the destination register.
2. RegWriteW -> WE3 (Write Enable): Controls whether the register file actually performs the write. Set to 1 for instructions that produce a result (R-type, I-type, Load, Jump). Set to 0 for Store and Branch instructions that do not write to a register.
3. RDW [4:0] -> A3 (Write Address): The 5-bit destination register number (rd field from the original instruction, carried through all pipeline stages via RD_E, RD_M, RD_W).

The write occurs on the FALLING edge of the clock (negedge clk) in the Register File, allowing the Decode stage to read the freshly written value on the RISING edge of the same cycle. This half-cycle write-then-read mechanism helps resolve Read After Write (RAW) hazards.

Writeback Examples

- add x7, x5, x6 (R-type):
ResultSrcW = 2'b00. ResultW = ALU_ResultW = 5 + 3 = 8. RegWriteW = 1. RDW = 7.
Register File writes 8 into x7 on negedge clk.

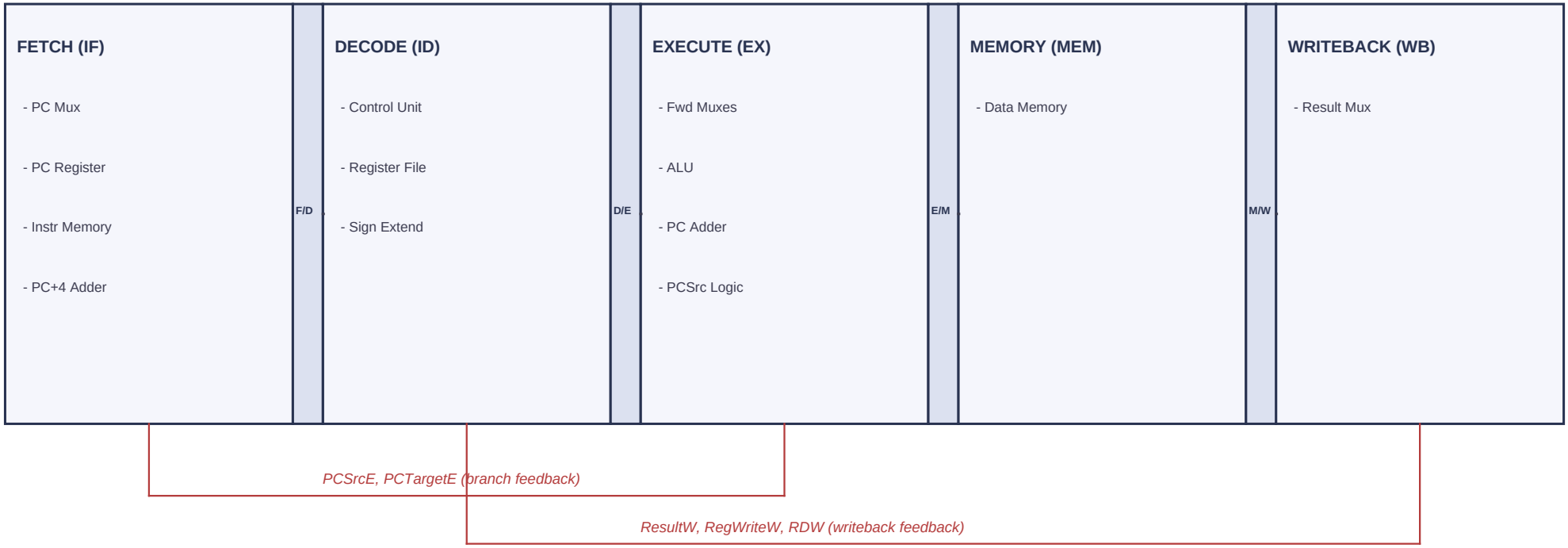
- lw x8, 0(x0) (Load):
ResultSrcW = 2'b01. ResultW = ReadDataW = mem[0] (the value loaded from address 0x00). RegWriteW = 1. RDW = 8.
Register File writes the loaded value into x8 on negedge clk.

- jal x1, target (Jump and Link):
ResultSrcW = 2'b10. ResultW = PCPlus4W = return address (PC of jal instruction + 4). RegWriteW = 1. RDW = 1.
Register File writes the return address into x1 (ra) on negedge clk. The program can later use 'jalr x0, x1, 0' to return.

Verilog Source Code: Writeback_Cycle.v

```
 1 | `ifndef WRITEBACK_CYCLE_V
 2 | `define WRITEBACK_CYCLE_V
 3 |
 4 | `include "Mux.v"
 5 |
 6 | module writeback_cycle (
 7 |     input clk,
 8 |     input rst,
 9 |     input [1:0] ResultSrcW,
10 |     input [31:0] PCPlus4W,
11 |     input [31:0] ALU_ResultW,
12 |     input [31:0] ReadDataW,
13 |     output [31:0] ResultW
14 | );
15 |
16 |     // 3-to-1 Multiplexer to select Writeback Result
17 |     Mux_3_by_1 result_mux (
18 |         .a(ALU_ResultW),
19 |         .b(ReadDataW),
20 |         .c(PCPlus4W),
21 |         .s(ResultSrcW),
22 |         .d(ResultW)
23 |     );
24 |
25 | endmodule
26 |
27 | `endif
```

Section 6: Top-Level Integration - Full Pipeline Block Diagram



Inter-Stage Wiring Table

This table summarizes every signal that crosses a pipeline stage boundary in Pipeline_Top.v.

From Stage	To Stage	Signal Name	Width	Purpose
Fetch	Decode	InstrD	32	Fetched instruction
Fetch	Decode	PCD	32	PC of the instruction
Fetch	Decode	PCPlus4D	32	Next sequential address
Decode	Execute	RegWriteE	1	Register write enable
Decode	Execute	ALUSrcE	1	ALU source B select
Decode	Execute	MemWriteE	1	Memory write enable
Decode	Execute	ResultSrcE [1:0]	2	Result source select
Decode	Execute	BranchE, JumpE	1+1	Branch/Jump control
Decode	Execute	ALUControlE [2:0]	3	ALU operation select
Decode	Execute	RD1_E, RD2_E	32+32	Register read data
Decode	Execute	Imm_Ext_E	32	Sign-extended immediate
Decode	Execute	RD_E, PCE, PCPlus4E	5+32+32	rd addr, PC, PC+4
Execute	Memory	RegWriteM, MemWriteM	1+1	Control signals
Execute	Memory	ResultSrcM [1:0]	2	Result source for WB
Execute	Memory	RD_M	5	Destination register addr
Execute	Memory	PCPlus4M, WriteDataM	32+32	PC+4, store data
Execute	Memory	ALU_ResultM	32	ALU result / mem address
Memory	Writeback	RegWriteW, ResultSrcW	1+2	Control for result mux + write
Memory	Writeback	RD_W (RDW)	5	Destination register
Memory	Writeback	PCPlus4W, ALU_ResultW	32+32	PC+4, ALU result
Memory	Writeback	ReadDataW	32	Data from memory

Execute	Fetch	PCSrcE, PCTargetE	1+32	Branch/jump redirect
Writeback	Decode	ResultW, RegWriteW, RDW	32+1+5	Register file writeback

Complete Example: addi x5, x0, 5 through ALL 5 Stages

Cycle-by-Cycle Trace of addi x5, x0, 5 (0x00500293)

Cycle	Stage	Key Activity	Key Signals
1	FETCH	PC=0x00; IMEM reads mem[0] = 0x00500293	InstrF=0x00500293, PCPlus4F=0x04, PCSrcE=0 => PC_F=0x04
2	DECODE	InstrD=0x00500293; Op=0010011 (I-type); rs1=x0, rd=x5	RegWriteD=1, ALUSrcD=1, ALUControlD=000, RD1_D=0, Imm=5
3	EXECUTE	ALU computes Src_A + Src_B = 0 + 5 = 5	Src_A=0 (x0), Src_B=5 (Imm), ResultE=5, PCSrcE=0
4	MEMORY	No memory access (MemWriteM=0); data passes through	ALU_ResultM=5, MemWriteM=0, RegWriteM=1, ResultSrcM=00
5	WRITEBACK	ResultSrcW=00 selects ALU_ResultW=5; writes to x5	ResultW=5, RegWriteW=1, RDW=5 => x5 = 5

Detailed Signal Flow:

Cycle 1 (Fetch): The PC register outputs PCF = 0x00000000. The Instruction Memory reads mem[0] = 0x00500293. The PC Adder computes PCPlus4F = 0x00000004. Since PCSrcE = 0 (from the Execute stage -- no branch), the PC Mux selects PCPlus4F = 0x04 as the next PC. At posedge clk, the F/D registers capture: InstrD = 0x00500293, PCD = 0x00, PCPlus4D = 0x04.

Cycle 2 (Decode): The Control Unit reads opcode 0010011 (I-type ALU) and generates: RegWriteD=1, ImmSrcD=00, ALUSrcD=1, MemWriteD=0, ResultSrcD=00, BranchD=0, JumpD=0, ALUOp=00, ALUControlD=000 (ADD). The Register File reads rs1=x0 -> RD1_D=0 and rs2=x5 -> RD2_D=0 (initially). Sign Extend produces Imm_Ext_D = 5. At posedge clk, D/E registers capture all values. RD_E = InstrD[11:7] = 5 (x5).

Cycle 3 (Execute): ForwardA_E = ForwardB_E = 00, so Src_A = RD1_E = 0 and Src_B_interim = RD2_E = 0. ALUSrcE = 1, so the ALU Source Mux selects Imm_Ext_E = 5 as Src_B. ALU computes ADD: 0 + 5 = 5. ResultE = 5. ZeroE = 0 (result is non-zero). BranchE = 0, JumpE = 0, so PCSrcE = 0. PCTargetE = PCE + Imm_Ext_E = 0x00 + 5 = 0x05 (unused since PCSrcE=0). At posedge clk, E/M registers capture: ALU_ResultM=5, RegWriteM=1, MemWriteM=0, ResultSrcM=00, RD_M=5.

Cycle 4 (Memory): MemWriteM = 0 so no write to Data Memory. ReadDataM is read but irrelevant (ResultSrcM=00). At posedge clk, M/W registers capture: ALU_ResultW=5, RegWriteW=1, ResultSrcW=00, RD_W=5, ReadDataW=mem[A>>2].

Cycle 5 (Writeback): ResultSrcW = 00 selects ALU_ResultW = 5. ResultW = 5. RegWriteW = 1 and RDW = 5, so the Register File writes ResultW = 5 into Register[5] (x5) on negedge clk. After this cycle, x5 = 5.

Verilog Source Code: Pipeline_Top.v

```
`ifndef PIPELINE_TOP_V
`define PIPELINE_TOP_V

`include "Fetch_Cycle.v"
`include "Decode_Cycle.v"
`include "Execute_Cycle.v"
`include "Memory_Cycle.v"
`include "Writeback_Cycle.v"

module Pipeline_top (
    input clk,
    input rst
);

    // Declaration of Interim Wires
    wire PCSrcE;
    wire RegWriteW, RegWriteE, ALUSrcE, MemWriteE, BranchE, JumpE;
    wire RegWriteM, MemWriteM;
    wire [1:0] ResultSrcE, ResultSrcM, ResultSrcW;
    wire [2:0] ALUControlE;
    wire [4:0] RD_E, RD_M, RDW;
    wire [31:0] PCTargetE, InstrD, PCD, PCPlus4D, ResultW, RD1_E, RD2_E, Imm_Ext_E, PCE, PCPlus4E, PCPlus4M, WriteDataM, ALU_ResultM;
    wire [31:0] PCPlus4W, ALU_ResultW, ReadDataW;

    // Forwarding Control Wires (Tied to 2'b00 since Hazard Unit is not instantiated)
    wire [1:0] ForwardAE, ForwardBE;
    assign ForwardAE = 2'b00;
    assign ForwardBE = 2'b00;

    // --- 1. Fetch Stage ---
    Fetch_Cycle Fetch (
        .clk(clk),
        .rst(rst),
        .PCSrcE(PCSrcE),
        .PCTargetE(PCTargetE),
        .InstrD(InstrD),
        .PCD(PCD),
        .PCPlus4D(PCPlus4D)
    );

    // --- 2. Decode Stage ---
    decode_cycle Decode (
        .clk(clk),
```

```
44 |         .rst(rst),
45 |         .InstrD(InstrD),
46 |         .PCD(PCD),
47 |         .PCPlus4D(PCPlus4D),
48 |         .RegWriteW(RegWriteW),
49 |         .RDW(RDW),
50 |         .ResultW(ResultW),
51 |         .RegWriteE(RegWriteE),
52 |         .ALUSrcE(ALUSrcE),
53 |         .MemWriteE(MemWriteE),
54 |         .ResultSrcE(ResultSrcE),
55 |         .BranchE(BranchE),
56 |         .JumpE(JumpE),
57 |         .ALUControlE(ALUControlE),
58 |         .RD1_E(RD1_E),
59 |         .RD2_E(RD2_E),
60 |         .Imm_Ext_E(Imm_Ext_E),
61 |         .RD_E(RD_E),
62 |         .PCE(PCE),
63 |         .PCPlus4E(PCPlus4E)
64 |     );
65 |
66 |     // --- 3. Execute Stage ---
67 |     execute_cycle Execute (
68 |         .clk(clk),
69 |         .rst(rst),
70 |         .RegWriteE(RegWriteE),
71 |         .ALUSrcE(ALUSrcE),
72 |         .MemWriteE(MemWriteE),
73 |         .ResultSrcE(ResultSrcE),
74 |         .BranchE(BranchE),
75 |         .JumpE(JumpE),
76 |         .ALUControlE(ALUControlE),
77 |         .RD1_E(RD1_E),
78 |         .RD2_E(RD2_E),
79 |         .Imm_Ext_E(Imm_Ext_E),
80 |         .RD_E(RD_E),
81 |         .PCE(PCE),
82 |         .PCPlus4E(PCPlus4E),
83 |         .PCSrcE(PCSrcE),
84 |         .PCTargetE(PCTargetE),
85 |         .RegWriteM(RegWriteM),
86 |         .MemWriteM(MemWriteM),
87 |         .ResultSrcM(ResultSrcM),
88 |         .RD_M(RD_M),
89 |         .PCPlus4M(PCPlus4M),
```

```
 90 |         .WriteDataM(WriteDataM),
 91 |         .ALU_ResultM(ALU_ResultM),
 92 |         .ResultW(ResultW),
 93 |         .ForwardA_E(ForwardAE),
 94 |         .ForwardB_E(ForwardBE)
 95 |     );
 96 |
 97 |     // --- 4. Memory Stage ---
 98 |     memory_cycle Memory (
 99 |         .clk(clk),
100 |         .rst(rst),
101 |         .RegWriteM(RegWriteM),
102 |         .MemWriteM(MemWriteM),
103 |         .ResultSrcM(ResultSrcM),
104 |         .RD_M(RD_M),
105 |         .PCPlus4M(PCPlus4M),
106 |         .WriteDataM(WriteDataM),
107 |         .ALU_ResultM(ALU_ResultM),
108 |         .RegWriteW(RegWriteW),
109 |         .ResultSrcW(ResultSrcW),
110 |         .RD_W(RDW),
111 |         .PCPlus4W(PCPlus4W),
112 |         .ALU_ResultW(ALU_ResultW),
113 |         .ReadDataW(ReadDataW)
114 |     );
115 |
116 |     // --- 5. Writeback Stage ---
117 |     writeback_cycle WriteBack (
118 |         .clk(clk),
119 |         .rst(rst),
120 |         .ResultSrcW(ResultSrcW),
121 |         .PCPlus4W(PCPlus4W),
122 |         .ALU_ResultW(ALU_ResultW),
123 |         .ReadDataW(ReadDataW),
124 |         .ResultW(ResultW)
125 |     );
126 |
127 | endmodule
128 |
129 | `endif
```